

处理器设计文档

404 NOT FOUND

黄思在 满洋 梁嘉俊

哈尔滨工业大学（深圳）

2022 年 8 月 18 日

目录

1 项目概述	3
2 处理器整体设计	4
3 SoC 设计	4
4 前端取指模块设计背景	5
4.1 取基本块结构	5
4.2 取指令流结构	5
4.3 Trace Cache 结构	5
5 前端取指模块设计与实现	5
5.1 总体设计结构	6
5.2 分支预测模块 BPU	6
5.2.1 取指目标缓冲 FTB	7
5.2.2 主预测器 TAGE	7
5.2.3 uBTB	8
5.3 取指目标队列 FTQ	8
5.3.1 预测块的生命周期	8
5.3.2 内部结构	8
5.4 取指模块 IFU	9
5.4.1 流水级	9
5.5 指令缓存 ICache	11
5.5.1 总体设计	11
5.5.2 状态机设计	11
5.5.3 流水级	11
5.5.4 替换策略	12
5.5.5 时序	12
5.6 返回地址栈 RAS	13
5.7 预译码模块 Predecoder	13
6 指令缓冲设计	14
6.1 规格	14
6.2 设计	14
7 译码模块设计	14
8 派遣和发射模块设计	14
8.1 发射逻辑	14
8.2 数据前递	15
9 执行阶段	15
9.1 乘除法器	15
10 寄存器堆设计	15
10.1 regs_file 模块	15
10.2 last_valid_table 模块	15
10.3 reg_lutram 模块	16

11 数据缓存模块	16
11.1 总体设计	16
11.2 缓存设计	16
11.2.1 地址编码	16
11.2.2 数据储存	16
11.2.3 状态机设计	16
11.2.4 流水级设计	17
11.2.5 dcache_fifo 模块	17
12 TLB 设计	18
12.1 TLB 模块	18
12.2 TLB Entry 模块	18
13 CTRL 模块设计	18
13.1 异常处理	18
13.1.1 实现的异常类型	18
13.1.2 异常处理流程	19
13.2 后端预测块提交信号	19
14 性能计数器	19
15 AXI 总线设计	20
15.1 读状态机	20
15.2 写状态机	21
15.3 仲裁器	21
15.3.1 写请求仲裁	21
15.3.2 读请求仲裁	21
16 系统展示	22
16.1 LCD 控制器	22
16.1.1 LCD 显示模块	22
16.1.2 LCD 触摸模块	23
16.1.3 字符绘制模块	23
16.2 软件展示	23
16.3 系统展示 soc	24
17 性能记录	25
17.1 Coremark 成绩	25
17.2 Fireye 测试套件成绩	25
17.3 Fireye 测试套件分支预测正确率	26
18 RoadMap	26
19 文档更新记录	27

1 项目概述

本项目是 2022 年龙芯杯，哈尔滨工业大学（深圳）404 NOT FOUND 队的参赛项目设计文档。

本处理器使用 SystemVerilog 进行开发，在仿真阶段使用 Verilator 进行仿真。使用了 Chiplab 平台提供的差分测试快速验证处理器的正确性。

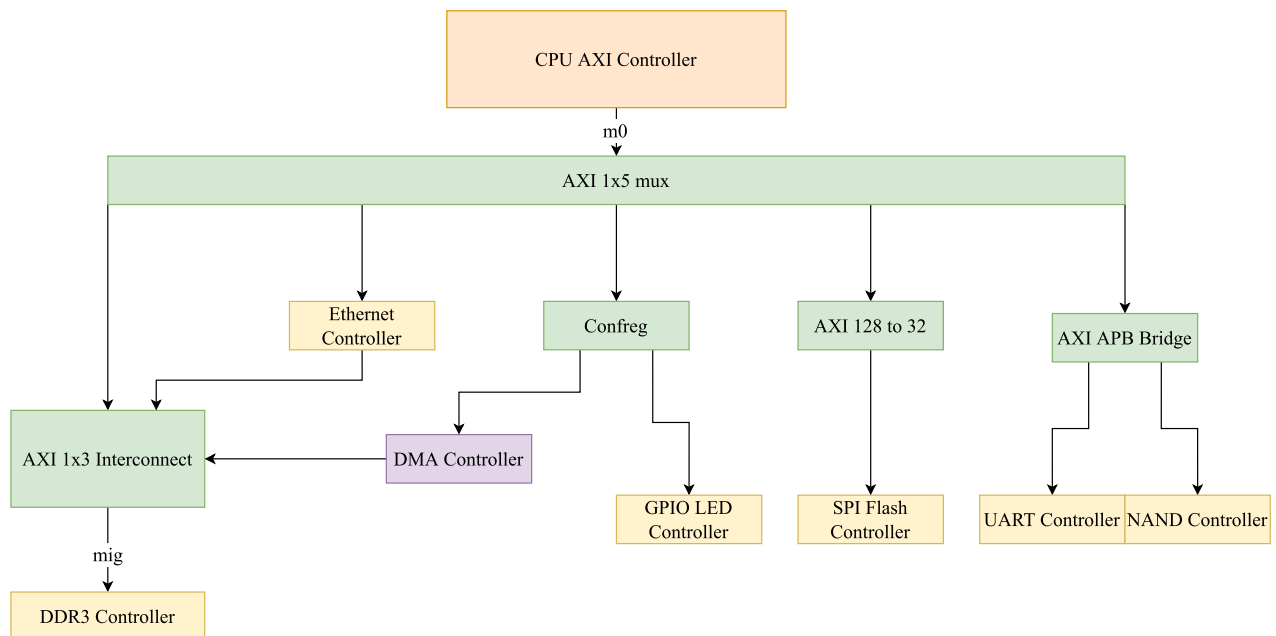
截止提交时，本处理器已经完成的功能有

1. 实现了除 IBAR, DBAR, CACOP 外的 **LoongArch32 Reduced** 指令集全部指令
2. 实现了 **10 级静态流水的超标量顺序双发射** 处理器
3. 实现了 L1i, L1d 的缓存操作指令 CACOP
4. 实现了具有两级分支预测器，返回地址栈，预译码的完整的现代处理器前端取指模块，取指能力接近龙芯 GS464E 内核。
5. 实现了 16-component TAGE 条件分支预测器，在 CBP5 的 trace 中可以取得 2.6 以下的 MPKI，在大部分性能测试项目中能够达到 96-99% 的预测正确率。
6. 实现了 2 路组相连的一级指令缓存，在性能测试中命中率达到 99% 以上
7. 实现了紧耦合的流水线一级数据缓存，能够满足 100MHz 主频
8. 实现了二级混合缓存，通过 AXI 中的 AxCACHE 信号能够实现部分缓存操作和数据一致性

在最终的提交版本上，我们的处理器在 coremark 测试中取得了 128 分的成绩，约为 1.8 coremark/MHz。在 Fireeye 测试套件中取得了 4.94 分的成绩。

在写回策略的一级数据缓存版本中，我们的处理器以 100MHz 在 coremark 中获得了 198 分的成绩，也即 1.98 coremark/MHz。超过了 ARM Cortex M1 核的水平。

<https://github.com/HITSZ-NSCSCC22/mycpu>



4 前端取指模块设计背景

为了适应处理器指令级并行 (ILP) 的需要, 为处理器执行单元提供充足和连续的指令流成为取指引擎迫切需要解决的任务。然而指令 Cache 失效、分支误预测和指令流中成功的分支跳转成为实现这个目标的巨大障碍。如何解决这个问题, 对处理器设计者来说, 是一个巨大的挑战。分离的前、后端设计就是为应对这样的挑战而提出的一种解决方案, 并已在现代高性能微处理器设计中被广泛采用 [?]

在提高指令前端的性能方面, 目前已有许多技术: 为提高指令传输性能, 提出了取基本块前端结构、取指令流前端结构和 Trace Cache 前端结构

4.1 取基本块结构

基本块: 由某个地址开始, 直到一个跳转的分支, 这其中的所有指令被称为一个基本块。其中可以包括从未跳转的条件分支

取基本块结构 (Fetch Basic Block) 是在论文 *A Scalable Front-End Architecture for Fast Instruction Delivery* [?] 中提出的。基本做法是在前端设置一个类 Cache 存储结构, 其中保存含有分支的基本块信息, 可以解决分支预测器端口数量和取指宽度之间的矛盾。

我们的处理器使用这一种前端结构设计。

4.2 取指令流结构

取指令流结构是取基本块结构的增强版, 在论文 *Fetching instruction streams* [?] 中提出。取指令流前端维护的类 Cache 存储结构不仅仅存储一个基本块的信息, 而是存储多个基本块的信息。这些基本块构成了一定长度的指令流, 因此称这种结构为取指令流结构。此种结构增大了一次分支预测能够产生的指令流长度, 因此可以提供更强的前端能力。

为了适应硬件存储的需要, 对存储的指令流通常有一定的限制, 例如最大长度, 其中含有的分支指令数量等等。

香山的南湖微架构版本使用此种结构 [?]

4.3 Trace Cache 结构

Trace Cache 结构是更加激进的取指令流结构, 处理器在前端直接保存全部的指令轨迹, 可以提高前端的取指能力。但是更新策略和匹配较为复杂, 因此只有某些商用处理器采用。

5 前端取指模块设计与实现

本文档中处理器前端指指令缓冲之前的模块, 包括

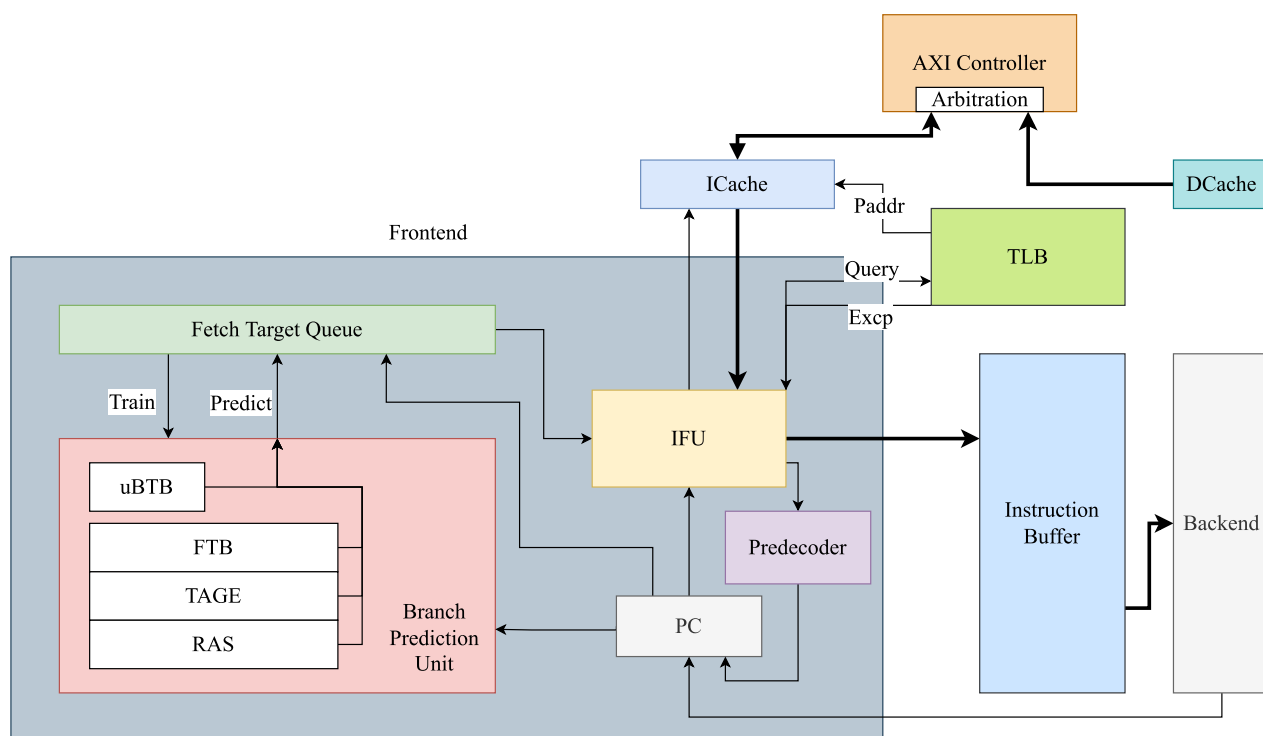
1. 取指模块 Instruction Fetching Unit (IFU)
2. 分支预测模块 Branch Prediction Unit (BPU)
3. 取指目标队列 Fetch Target Queue (FTQ)
4. 指令缓存 ICache
5. 返回地址栈 Return Address Stack (RAS)
6. 预译码模块 Predecoder

名词解释

预测块: 分支预测单元 (BPU) 每次给取指目标队列 (FTQ) 的请求基本单位, 它描述了一个取指请求的范围

我们的处理器中, 前端取指模块的取指能力设计为最大 4 条指令。

5.1 总体设计结构



5.2 分支预测模块 BPU

BPU 每周期最多预测一个地址，也最多给出一个对应于该地址的预测块

BPU 保证给出的预测块不跨页，这是因为 TLB 的 ICache 查询端口每周期仅能查询一项，所以为了保证 TLB 的查询结果对于预测块内的所有指令都有效，需要保证这些指令不跨页

我们的前端模块存在有"Taken Bubble"问题，但是由于前端能力强于后端，因此没有必要继续增加前端的复杂度。

BPU 有以下子模块：

1. PC：保存当前预测基地址，这个 PC 只可能是某个预测块的结束（达到最大取指宽度或一个分支的目标）
2. uBTB：用于提供一个当拍的结果，大多数时候给出一个等于取指宽度的结果
3. FTB：用于存储预测块的情况，相当于传统 BPU 中的 BTB
4. TAGE：主预测器，2 拍返回结果

PC 的更新逻辑

优先级从上到下递减：

后端的重定向

预译码的重定向

主预测器的覆盖重定向

uBTB 查询的结果

$PC + \text{取指宽度} * 4\text{Bytes}$

BPU 流水

L0 使用当前 PC 查询 uBTB、FTB 和 TAGE 预测器，uBTB 当拍返回，生成发送给 FTQ 的预测块

L1 预测块发送至 FTQ。FTB 和 TAGE 查询返回，如果命中且与 uBTB 产生的预测块有差异，生成前端重定向信号和新的预测块

BPU 与 FTQ 交互逻辑

L0 和 L1 都与 FTQ 直接连接，每周分别发送预测块和前端重定向信号

如果 FTQ 满，那么将所有输出给 FTQ 的信号全部拉低

5.2.1 取指目标缓冲 FTB

取指目标缓冲 Fetch Target Buffer (FTB) 主要用于存储预测块的信息 [?], 因为需要存储的信息很多, 使用 BRAM 实现

FTB 使用预测块的开始虚地址 PC 进行索引。设计为 4-way associated, 替换策略为随机替换
FTB 规格:

1. 4-way associated
2. 1K entry per way, 4K entry in total
3. Pseudo random replace policy

FTB 的表项与预测块的开始地址一一对应, 也即是说用于查询和更新的 PC 仅可能是两种情况之一: 1. 与取指宽度对齐的 PC, 且取指宽度内有发生跳转的分支 2. 分支的 fall-through address

FTB 表项内容

1. tag
2. fall-through address
3. jump-target address
4. branch type
5. cross cacheline

表项的更新 处理器初始化时, 整个 FTB 都为空, 所以初始化后的最初若干指令并不会通过 FTB, 而是通过 uBTB 或者直接 + 取指宽度进行预测。

FTB 表项更新的时机: 考虑到 TAGE 预测器对性能测试负载的分支信息更新及时性不是很敏感, 而 FTB 作为整个前端推测执行的基础, 正确性非常重要, 因此只有在后端提交分支指令的时候才考虑更新 FTB。FTB 表项更新的条件:

1. 非条件分支
2. 未在 FTB 中出现、发生了跳转的条件分支
3. 在 FTB 中出现, 但是跳转信息错误的分支

后端返回的信息包括: FTQ ID、发生分支的指令在预测块中的位置、分支的类型

5.2.2 主预测器 TAGE

主预测器规格:

1. 16-component TAGE
2. 1K entry, 13bits tag, 3bits ctr, 3bits usefull Tagged Predictor
3. 16K entry, 2bits ctr Base Predictor
4. 32 entry, 4bits global use_alt counter

主预测器 TAGE 的更新条件：

1. 分支在 FTB 中出现，同时在后端提交，更新表项
2. 分支未在 FTB 出现，但是发生了跳转，初始化对应的表项

5.2.3 uBTB

使用寄存器堆实现，查询当拍返回，每次可查询的地址数量等于最大取指宽度

5.3 取指目标队列 FTQ

FTQ 是分支预测和取指单元之间的缓冲队列，它的主要职能是暂存 BPU 预测的取指目标，并根据这些取指目标给 IFU 发送取指请求。它的另一重要职能是暂存 BPU 各个预测器的预测信息，在指令提交后把这些信息送回 BPU 用作预测器的训练，因此它需要维护指令从预测到提交的完整生命周期。

香山实现，我们暂未实现：由于后端存储 PC 的开销较大，当后端需要指令 PC 的时候，会到 FTQ 读取。目前已经实现将 FTQ ID 传入后端，而不是传送所有分支预测信息接入分支预测的时候将会参考香山的实现。

一般 FTQ 并不会很大，因为许多信息需要当拍读出，所以大部分关键信息将会使用寄存器堆实现。

在我们的处理器中，FTQ 的大小为 16

5.3.1 预测块的生命周期

指令以预测块为单位，从 BPU 预测后便送进 FTQ，直到指令所在的预测块中的**所有指令全部在后端提交完成**，FTQ 才会在存储结构中完全释放该预测块所对应的项。这个过程中发生的事如下 [?]:

1. 预测块从 BPU 发出，进入 FTQ，bpuPtr 指针加一，初始化对应 FTQ 项的各种状态，把各种预测信息写入存储结构；如果预测块来自 BPU 覆盖预测逻辑，则恢复 bpuPtr 和 ifuPtr
2. FTQ 向 IFU 发出取指请求，ifuPtr 指针加一
3. 指令进入后端执行，如果后端检测出了误预测，则通知 FTQ，给 IFU 和 BPU 发送重定向请求，恢复 bpuPtr 和 ifuPtr 到重定向指令对应的 FTQ ID (如果分支重定向，那么恢复到 EX 级的 FTQ ID，如果异常等重定向，恢复到 WB 级的 FTQ ID)
4. 指令在后端提交，通知 FTQ，等 FTQ 项中所有的有效指令都已提交，commPtr 指针加一，从存储结构中读出相应的信息，送给 BPU 进行训练

5.3.2 内部结构

表的种类：

1. pc_table 寄存器堆实现，用于存储指令地址相关的内容，从 BPU 接收信息
2. bpu_meta_table lutram 实现，存储 BPU 相关信息

pc_table 表项内容

1. start_addr 预测块的起始地址
2. is_cross_cacheline 表示是否跨缓存行
3. length 长度，表示该预测块的长度，IFU 使用这一信息向 IB 发送指令
4. ftq_id FTQ ID，用于后端提交指令或进行重定向的时候恢复 FTQ 指针

bpu_meta_table 表项内容

- 1. 分支预测器的 meta 信息
- 2. 后端 EX 级返回的分支地址

5.4 取指模块 IFU

取指模块 IFU 负责从 FTQ 中获取预测块，然后向 ICache 发送取指请求，并将 ICache 返回的有效指令送入指令缓冲 IB [?]

取指访存类型的生成：

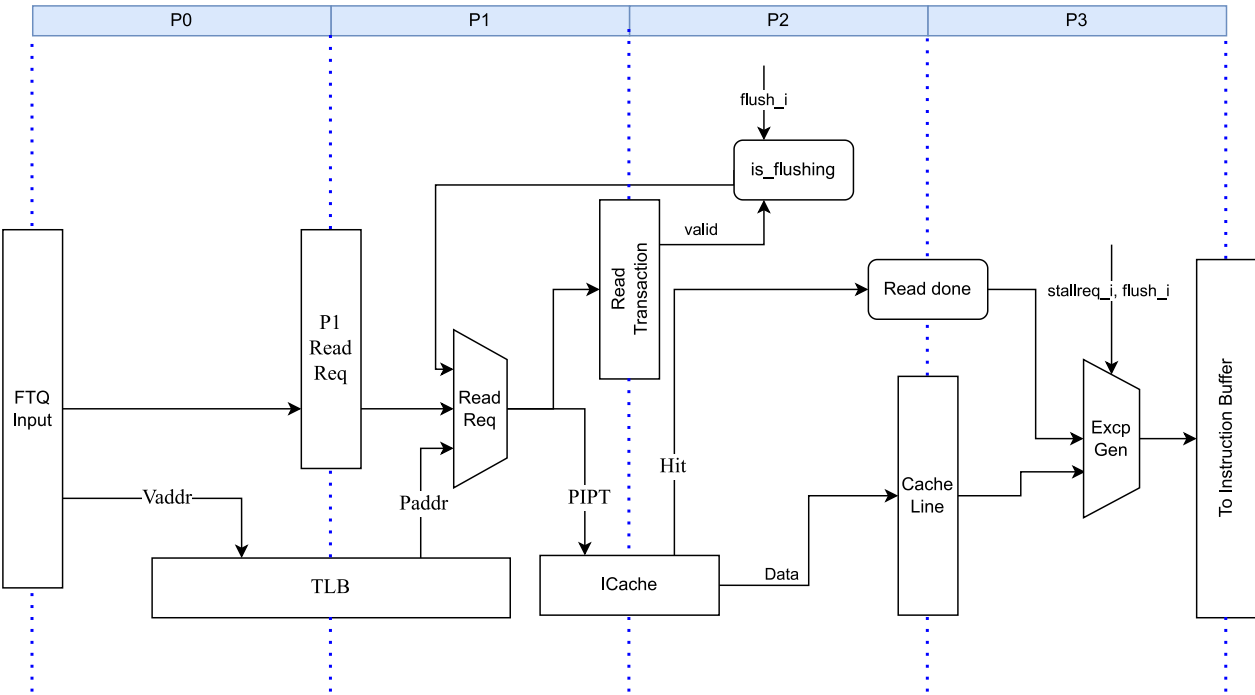
- (1) 如果处理器处在直接地址翻译模式，访存类型来自 CSR.CRMD.DATF
- (2) 如果处理器处在映射地址翻译模式，且落在 DMW 中，访存类型来自 DMW 的配置
- (3) 如果处理器处在映射地址翻译模式，且不在 DMW 中，访存类型来自 TLB 中 MAT 域

IFU 可能遇到的几种情况：

- 1. ICache 1 周期返回了有效的数据
- 2. ICache miss，不定周期返回有效的数据
- 3. IB 满，需要暂停
- 4. 收到后端的冲刷信号，需要刷空并且等待 ICache 返回有效

以及他们互相之间的组合

5.4.1 流水级



P0

从 FTQ 获取信息，包括 FTQ ID 和 start_pc 等信息。
持续向 TLB 发送查询请求

设置 p0_advance 信号, 该信号拉高时表示 p0 阶段的信息可以传入 p1

所以 p0_advance 同 p1_advance

同周期向 FTQ 返回是否接受输入, 条件同 p0_advance

P1

本周期收到从 TLB 返回的物理地址和是否命中等信息。同周期向 ICache 送去取指请求, 取指请求使用的是物理地址。发送请求的条件如下:

1. p1_advance
2. 请求的地址是合法的, 即 TLB 命中或者在 DMW 中

p1_advance 的条件如下:

1. 没有冲刷信号, 没有 IB 满停顿信号
2. P0 阶段的 FTQ block 是有效的
3. P2 不在读事务中

p1_advance 拉高时, P1 阶段的请求被移入 P2

P2

P2 阶段的寄存器包括有:

1. FTQ ID
2. FTQ block (预测块)
3. CSR 信息 (P0 阶段的 CSR 信息)
4. TLB 相关信息

有可能当周期从 ICache 拿到指令数据, 也有可能不定周期从 ICache 拿到数据

因此维护一个与 ICache 返回同拍的读事务完成信号, 如果读事务未完成, 要求 P1 停顿/不发出请求

因为 ICache 的两个端口可能不同周期返回数据, 因此维护一个寄存器暂存返回有效信号, 同时用于生成读事务完成

如果有冲刷信号且有正在进行的读事务, 将冲刷信号写入暂存寄存器, 并维护一个同拍有效并维持的信号, 直到读事务完成, 然后将输出刷空。如果没有正在进行的读事务, 那么冲刷信号仅有一拍

p2_advance 信号与 p2_read_done 信号相同

P3

将指令送入 IB, 在一些额外信息附加到指令信息中, 包括:

1. FTQ ID
2. FTQ Index
3. 四种异常信息

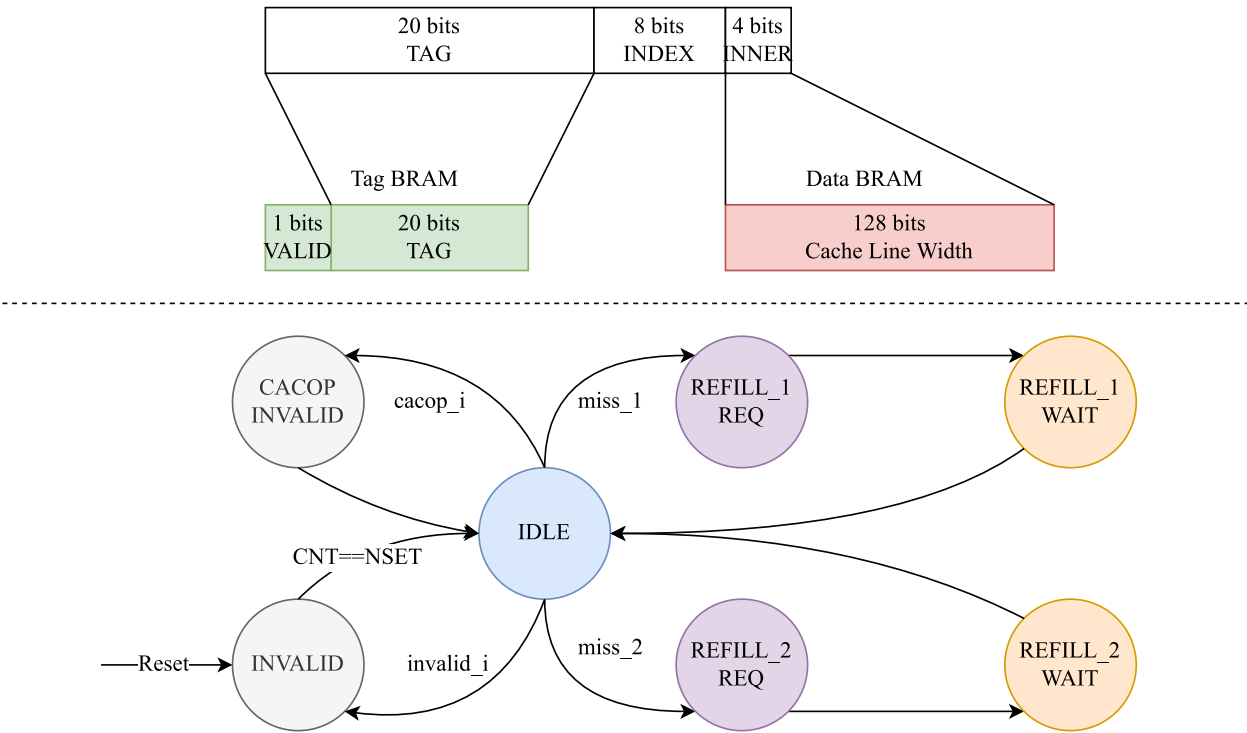
如果有冲刷信号, 将输出刷空。

如果有 IB 满停顿信号, 维持输出

如果 P1 的读事务未完成, 输为 0

5.5 指令缓存 ICache

5.5.1 总体设计



使用可参数化的设计，支持配置组相连路数和大小
存储部分使用 BRAM 实现，最大每个 bank 使用 32K。
替换策略使用随机替换
不设置冲刷信号，如果产生重定向，由 IFU 控制冲刷，ICache 保证会最终返回 rvalid

5.5.2 状态机设计

上电复位后，ICache 进入 INVALID 状态，然后硬件复位所有 TAG BRAM，完成后进入 IDLE 状态。
IDLE 状态下可以接受读请求和 Cache 操作控制信号。优先级从前到后如下：刷空控制信号，CACOP 控制信号，读请求。如果接收到 Cache 刷空操作信号，进入 INVALID 状态并刷空 TAG RAM。如果接收到 CACOP 控制信号，进入 CACOP INVALID，按照手册要求刷空某一行，下一周期返回 IDLE。如果接收到读请求，接收到的下一周期 BRAM 返回结果，然后同时产生 miss 信号，如果 miss 发生，则进入 REFILL_REQ 状态，如果同时发生 miss，优先重填端口 1 的请求，如果两个端口都命中，则停留在 IDLE
在 REFILL_REQ 状态下，向 AXI 控制器发送读请求，下一周期进入 REFILL_WAIT 状态。此状态应当拿到 TLB 的查询结果，因此向 AXI 发送的请求使用的是物理地址
在 REFILL_WAIT 状态下等待 AXI 控制器返回 rvalid 信号。rvalid 信号拉高时，BRAM 对应端口的写使能拉高，将 AXI 控制器传来的数据送到 BRAM 的写端口。TAG 使用从 TLB 传来的物理 TAG。下一周期返回 IDLE 状态

5.5.3 流水级

P0

将输入的地址送入 BRAM 查询

P1

BRAM 返回结果，对比 tag 等判断是否命中

本周期已经从 TLB 拿到查询结果，因此使用物理 TAG 进行对比
miss 信号的生成逻辑：

1. 如果处于 IDLE 状态，那么对比 BRAM 返回的 tag，判断是否命中
2. 如果处于任意重填状态，那么 miss 一直拉高，直到 AXI 返回 valid 信号

如果 miss 进入其中一个重填状态

使用 p1 前缀的几个变量存储上一周期从 P0 送入的变量，这些变量的变化逻辑：

1. 如果 P1 发现未命中，那么保持这些变量不变，如果命中，拉取 P0 输入的变量
2. 如果处于重填状态，接到 AXI 返回的对应 valid 信号，将这些变量清空

5.5.4 替换策略

使用随机替换，使用一个 LFSR 寄存器产生伪随机数，当缓存缺失时，随机选择一个 bank 进行替换

5.5.5 时序

总体原则：

任意时刻都可以发送读请求，但是不保证该读请求被接受，读请求被接受的当周期会返回握手信号。也就是说 IFU 需要保持读请求信号直到 ICACHE 的确认信号拉高的下一个时钟上升沿。

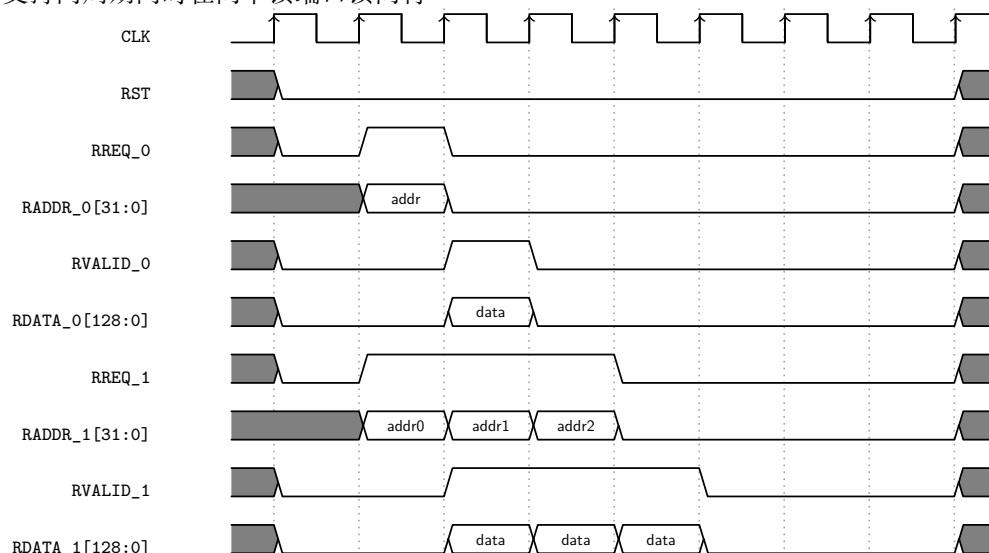
读请求如果被接受，那么 ICACHE 一定会返回 rvalid。

rvalid 拉高的同周期可以发送下一个读请求。

读命中时序

读请求如果命中，下一周期给出结果同时接受新的读请求。

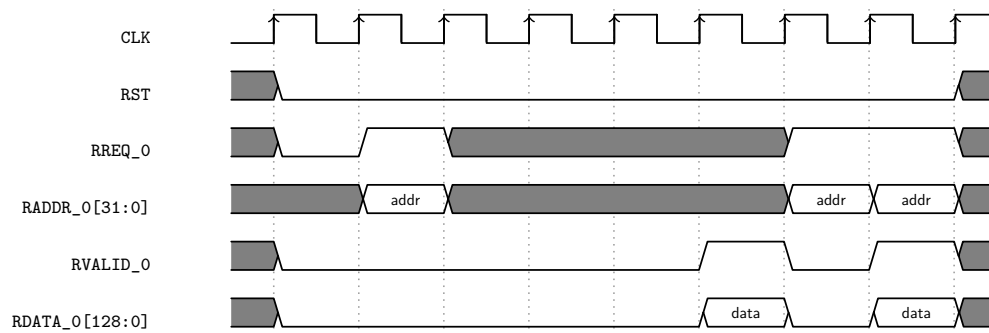
支持同周期同时在两个读端口读两行



读缺失时序

读请求的概念是同一周期的 RREQ 和 RADDR，如果发生读缺失，后续的任何读请求都被忽略

两个端口的“读请求”是独立的，如果两个端口都缺失，那么返回有效数据的延迟都是不确定的，也不保证（大概率不会是）同一拍



5.6 返回地址栈 RAS

分支预测单元内部实现有返回地址栈 (RAS)，当有函数调用指令在后端提交时，将 fall-through address 压入 RAS，当后端提交函数返回指令时，将栈顶元素弹出。没有实现推测更新。

预测时根据 FTB 结果判断是否可能为函数调用返回，如果是则采用 RAS 的结果。

函数调用的指令

1. BL
2. JIRL \$r1, \$rX, 0

函数返回的指令

1. JIRL \$r0, \$r1, 0

5.7 预译码模块 Predecoder

指令从取指单元 (IFU) 送出的同时进入预译码器进行预译码，如果预译码器检测出了预测错误或者没有给出预测的非条件分支，则会更新送出的指令信息，并产生一次预译码器重定向。

6 指令缓冲设计

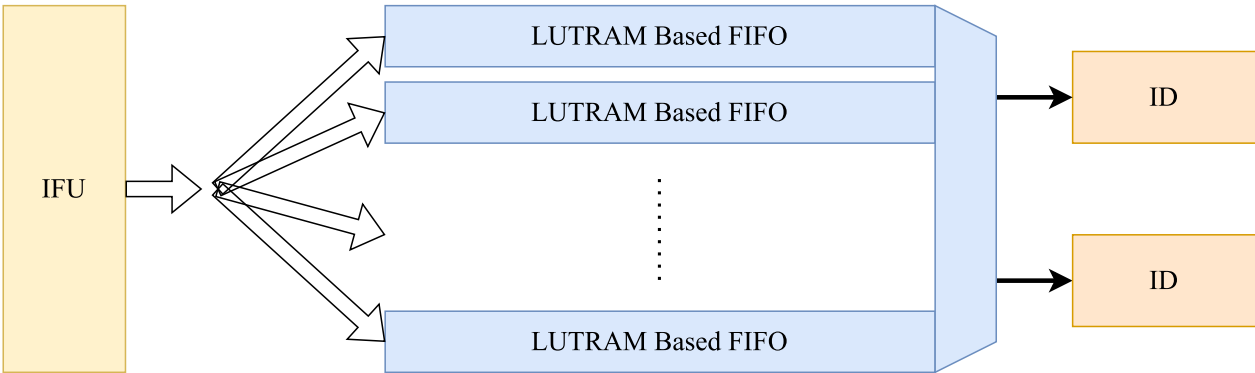
指令缓冲 (Instruction Buffer) 是存在于前端和后端之间的缓冲区，用于避免前端取指阻塞和后端访存阻塞之间的互相影响。

6.1 规格

由于改用 LUTRAM 实现的 FIFO 占用 FPGA 资源很少，因此我们选择了充分大的指令缓冲，在设计中使用了 32 entry 的指令缓冲。

6.2 设计

我们的设计参考了 2019 年龙芯杯清华大学编程是一件很危险的事情队的设计 [?], 使用多个 Bank 的一读一写 FIFO 和输入输出两个指针，构成了多人多出的 LUTRAM Based FIFO



7 译码模块设计

译码模块功能：

1. 译码，将指令转化为指令信息
2. 获取中断信号，如果发生中断，那么将中断信息附加到指令上

因为 la32 指令集中的指令码并非定长，所以我们根据指令码的长度，即不同的指令类型，将指令分别在 10 个模块中进行译码，然后再在顶层的 id 模块中进行整合，不同指令码的前缀会保证一条指令不存在二义性。若一条指令没有被翻译为任何一种指令，则会触发无效指令异常。

8 派遣和发射模块设计

8.1 发射逻辑

dispatch 模块用于对 id 模块传来的指令进行仲裁和发射。其发射逻辑如下：

单发射情况

1. 两条指令存在数据冲突时，只发射前面的指令
2. 当两条指令中存在访存指令时，两条指令均为单发射
3. 当两条指令中存在特权指令时，两条指令均为单发射

暂停情况

1. dispatch 为空，原因可能是前端发生重定向导致流水线刷空，或 icache 重填
2. 后端的暂停信号，当遇到乘除法指令或访存未命中

3. 存在数据冲突，常见于 load to use

其余状态下均为双发射

8.2 数据前递

所有的数据前递都在 dispatch 阶段解决

dispatch 阶段实现了一个寄存器占用记分板，当一条要写入寄存器的指令被发射时，对应的位置会被置为 0，表示该寄存器数值不是最新的值，直到后端完成运算或 load 请求，才会把对应的位置拉高，表示寄存器的值可以使用。

同时，dispatch 阶段会按照顺序从寄存器堆和前递的数据中选择数据，优先级为 ex->mem1->mem2->wb->regs

9 执行阶段

执行阶段进行逻辑运算指令，移位运算指令和算数运算指令的处理。如果有访存请求，执行阶段也会进行虚地址的计算并发送请求，同时，查询 tlb 的请求也在此阶段发送。

执行阶段还处理最终的分支结果，如果跳转方向或者跳转地址出现了预测错误，将会触发一个后端重定向。

9.1 乘除法器

在两条流水线的执行模块下，均实例化了一个乘法器和一个除法器，可以实现乘除法指令的双发射

在对乘法器的选择上，我们放弃了传统的华莱士树，选择了 fpga 板自带的乘法器资源 dsp48，通过加入一级流水的方式，让 dsp48 能够使用完整的一个周期计算出结果，总共花费 2 周期完成计算，加上暂停结束的延迟就是 3 周期完成。

除法器上，我们使用了论文 *Rethinking Integer Divider Design for FPGA-Based Soft-Processors* [?] 中实现的除法器。这个除法器的原理是通过计算 clz(counting lead zero)，来减少除法计算的周期。除法平均需要的周期数量为 3.69 [?]

10 寄存器堆设计

通常情况下，寄存器堆是直接由一个 32*32 的 reg 数组实现，并进行多读多写。然而这样的实现方式会使得在上板实现时使用大量资源，同时不利于实现工具进行布线。故我们参考了开源 RISC-V 核心 cva5[?] 使用 Live Value Table (LVT) 的方式，使得在实现时可以使用 lutram 作为存储资源而非 reg，从而来设计一种 fpga 友好的寄存器堆模块

10.1 regs_file 模块

regs_file 为寄存器堆的顶层模块，用于与外界交互，接受外界的读写请求并进行相应的响应

regs_file 模块共有两个写端口和四个读端口，以支持双发射核心

regs_file 内部有 8 个 reg_lutram 模块，其中一个写端口对应 4 个 reg_lutram，一个读端口对应 2 个 reg_lutram，当有写请求时，对应的 4 个 reg_lutram 被同时写入，当有读请求时，会同时读出两个数据并根据 last_valid_table 的值进行选取

regs_file 内部有一个 last_valid_table 模块，用于判断选取读出的值

10.2 last_valid_table 模块

last_valid_table 用于当一个读端口同时会读取两个数据时判断该选择哪个数据。

last_valid_table 有两个写端口和 4 个读端口，内部有一个 32 位的 reg 数组，当有新的数据写入时，last_valid_table 中数组对应的位置也会被更新，若位第一个端口写入，则更新为 0，若位第二个端口写入，则更新为 1。

10.3 reg_lutram 模块

reg_lutram 模块是寄存器堆的储存模块，有一个读端口和一个写端口，内部有一个 32*32 的寄存器数组。只有一个写端口的设计使得其在上板是可以使用 lut 资源进行实现

11 数据缓存模块

11.1 总体设计

使用可参数化的设计，支持配置组相连路数和大小
 存储部分使用 BRAM 实现，最大每个 bank 使用 32K。
 写回策略使用 write-back 和 write-allcate 策略
 替换策略使用随机替换

11.2 缓存设计

11.2.1 地址编码

32 位虚拟被分为：
 Tag 位（物理地址高位）：20 位 Index 位（组地址）：8 位 Offset 位（块内偏移）：4 位
 32 位物理被分为：
 Tag 位（物理地址高位）：大于 20 位 Index 位（组地址）：小于 8 位 Offset 位（块内偏移）：4 位
 这是因为我们使用了 VIPT 的查询方式
 对于额外的标志位，有以下的设定。
 Valid 位（数据是否有效）：1 位 Dirty 位（脏标志）：一位
 其中 TAG_WIDTH 位 tag + 1 位 valid + 1 位 dirty 共 TAG_WIDTH + 2 位，用 tag_bram 储存

11.2.2 数据储存

Dcache 中的每个 cacheline 共 4 个字，用 data_bram 储存

11.2.3 状态机设计

初始化后进入 IDLE 状态，在此状态下表示通道空闲

IDLE 状态下可以接受读写请求和 Cache 操作控制信号。如果接收到 CACOP 控制信号，根据手册要求和具体指令进行查找和无效化，并将数据存入 fifo。如果接收到读写请求，会在第三周期判断是否为可缓存请求已经是否命中，根据不同情况直接返回或是进入不同处理状态如果发现 FIFO 被填满，则此时会暂停流水并进入 FIFO_CLEAR 状态，在 FIFO 被清空后才返回

READ_REQ,WRITE_REQ,UNCACHE_READ_REQ,UNCACHE_WRITE_REQ:

保持当前状态直到 axi 空闲，然后向总线发送读写请求，同时进入等待状态

READ_WAIT,WRITE_WAIT,UNCACHE_READ_WAIT,UNCACHE_WRITE_WAIT:

保持当前状态直到 axi 返回读写完成信号，然后根据具体状态决定是否返回读数据，以及是否要将相关的信息写入缓存

FIFO_CLEAR:

此阶段下，dcache 流水暂停且不会往 axi 发送任何请求，仅有 FIFO 可以发送写请求，当 FIFO 被清空后，返回 IDLE 状态

11.2.4 流水级设计

流水级的目的是使得 dcache 在命中的情况下可以不停顿，同时不影响核心的频率

P1:

在 ex 阶段，如果有访存指令或 cacop 指令，则会将计算出来的虚地址送往以及读写请求，选通字节等信息送往 dcache，查询 tag 和 bram，同时将相关信息送往 tlb，查询物理地址。

P2:

在 mem1 阶段，在 dcache 内，送入的物理地址会和从 bram 中读出来的 tag 进行对比，得到 tag_hit 信号，同时会将 paddr 送往 fifo 查询在下一个周期得到 fifo_hit 信号

P3:

在 mem2 阶段，对于可缓存请求，会得到 fifo_hit 信号，并整合得到 hit 信号，

tag/fifo 命中时，若为读请求，则将读取的数据返回，若为写请求，则将写入数据准备好，下一个周期写入 bram/fifo

对于不可缓存请求和未命中的缓存请求，我们将暂停流水线，同时判断 mem2 和 wb 传来的异常信息，确认请求可以发送后，dcache 进入重填状态并持续等待直到重填完成

11.2.5 dcache_fifo 模块

dcache_fifo 用于暂存写回的数据，并在 axi 空闲时将其写回，从而减少流水线的暂停以提高效率

dcache_fifo 中有一个 cpu 写端口和 cpu 读端口，用于和 cpu 进行读写的交互，同时还有一个 axi 写端口，用于将数据送回 axi

我们注意到 fifo 的写来源有且仅有一个，所以我们将 FIFO 的数据储存队列用 lutram 实现

写入 FIFO 的数据来源：写命中 FIFO：此时将 FIFO 读出命中的块与写数据合并送入 FIFO 再次进行覆盖写入。脏块弹出：将随机数指向的数据送入 FIFO 写入，但是要注意的是 AXI 取出的输出写入了 ram 之后，脏块就丢失了，因此需要在 AXI 取数据的时候将 ram 中的块拿出来，等待时钟沿的到来。

FIFO 写冲突：特别地，对于 WriteBuffer，我们注意到写替换命中会出现冲突，当写命中的数据正在被写入总线，就可能会使得我们想写入的数据没有进入总线。对此，在写入时进行判断，如果正准备被写入 axi 的地址同时被 cpu 重新写入，那么我们把 axi_wen_o 设为低位，同时保持 queue_valid 的值不变，并让头指针后移，使得此数据暂不写回，在下一个循环才写回。

FIFO 满处理：FIFO 只有在一次写入后才可能为满，而我们注意到，对 fifo 真正的写入只会发生在 IDLE 状态（重填结束的下一周期和 cacop 结束的下一周期），又因为 IDLE 状态下 dcache 不会向 axi 发送请求，所以一旦我们检测到 fifo 为满，我们立刻暂停流水线，同时切换至 FIFO_CLEAR 状态，让 fifo 得以清空。

遗憾的是，我们的 dcache 在启动 Linux 时，遇见了一个无法解决的问题：在启动 pmon 对 nand 进行扫描时，我们有一定概率检测到 1024 个坏块，即将整个 nand 检测为坏块直到比赛提交截止的当天，我们也没有办法解决这个问题。所以我们使用了来源于 Github 的 cache IP 和 Xilinx 的 System Cache Ip 作为替代，因其不是我们自己实现，下面对其进行简单介绍

L1 cache:iob-cache 来源于 Github 的开源 IP cache，写直通 cache，二路组相联，32K

L2 cache:Xilinx system cache IP 写回式 cache，四路组相联，512K

因为这两个 cache 并非我们实现，只是将其作为 IP 使用，我们也不了解其实现原理，故在此不对其做详细介绍

12 TLB 设计

TLB 共 32 个表项，设计为全相联的组织模式
TLB 表项构成

VPPN	Page Size	G Flag	ASID	E Flag
PPN0	PLV0	MAT0	Dirty Flag	Valid Falg
PPN1	PLV1	MAT1	Dirty Flag	Valid Falg

12.1 TLB 模块

TLB 模块是 TLB 顶层模块，与 CPU 核中其余部分交互。
根据传入的地址翻译模式是否打开，和各种 CSR 信息，判断是否采用 TLB Entry 中的信息，生成最终的查询结果 paddr。该结果即为地址翻译的最终结果，共取指和访存相关模块使用

12.2 TLB Entry 模块

TLB Entry 模块有两个查询端口 s0 和 s1，分别为指令和 mem 级使用。一个读端口和一个写端口，为 TLB 读写指令使用。读端口是当拍返回，写端口同步写入。两个查询端口均是下一周期返回结果
一个 TLB 失效端口，为 TLBINV 指令使用。判断失效指令的逻辑在本模块中，具体见于手册中
此外 TLB Entry 模块中还集成了两个 one_hot_to_index 模块，用于将第一周期查找所得出的 32 位的查找结果转化为一个 5 位的结果，然后利用此结果便可以直接取出对应表项

13 CTRL 模块设计

CTRL 模块是控制模块，含有以下职责：

- 1. 暂停信号的生成
- 2. 指令提交的仲裁和异常的处理
- 3. 生成后端预测块提交信号

13.1 异常处理

处理器在运行的过程中会遇到一些特殊情况，这时候打断原有的执行流程对其进行处理，这就是异常。

13.1.1 实现的异常类型

我们实现了大赛要求的所有异常，如下表所示

Ecode	EsubCode	例外代号	例外类型
0x0	0	INT	中断。
0x1	0	PIL	load 操作页无效例外

Ecode	EsubCode	例外代号	例外类型
0x2	0	PIS	store 操作页无效例外
0x3	0	PIF	取指操作页无效例外
0x4	0	PME	页修改例外
0x7	0	PPI	页特权等级不合规例外
0x8	0	ADEF	取指地址错例外
	1	ADEM	访存指令地址错例外
0x9	0	ALE	地址非对齐例外
0xB	0	SYS	系统调用例外
0xC	0	BRK	断点例外
0xD	0	INE	指令不存在例外
0xE	0	IPE	指令特权等级错例外
0xF	0	FPD	浮点指令未使能例外
0x12	0	FPE	基础浮点指令例外
0x1A-0x3E			保留编码
0x3F	0	TLBR	TLB 重填例外

13.1.2 异常处理流程

LoongArch 中的异常为“精确异常”，所谓精确异常，就是对应发生异常的指令，位于该指令前的所有指令都要被执行，其后的指令不被执行。为实现精确异常，对流水线中发生的异常，不立刻进行处理，而是进行记录，在到达 CTRL 部件后统一处理

13.2 后端预测块提交信号

后端预测块提交信号的生成有以下原则：

- 1. 如果某条指令携带有异常，那么该指令必然是基本块中的最后一条指令，拉高提交信号
- 2. 如果某条指令在 EX 级发生了分支重定向，那么该指令也必然是最后一条指令，拉盖对应的提交信号
- 3. 如果某条指令携带有 last 信号，那么该指令是 FTQ 预测块中的最后一条指令，拉高提交信号

以上原则都是在指令有效的前提下，如果由于异常等原因导致指令是无效的，那么他们的提交信号当然也是无效的

14 性能计数器

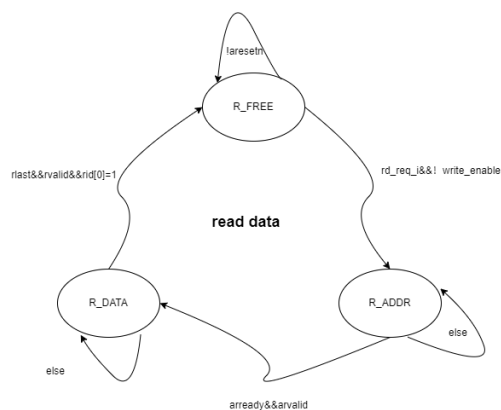
核内在 CSR 实现了一系列非标准性能计数器，用于量化分析处理器各部分的情况。
为了在用户态运行性能测试程序的同时可以读取性能计数器，在调试阶段时关闭处理器的特权等级检查。

CSR 编号	计数器内容
0x200	指令缓冲满计数
0x201	Dispatch 后端引起的空泡计数
0x202	Dispatch 前端引起的空泡计数
0x203	Dispatch 单发射指令引起的空泡计数
0x204	Dispatch 数据依赖引起的空泡计数
0x205	Dispatch 发射指令计数
0x206	指令缓冲空计数
0x210	ICache 请求计数
0x211	ICache 缺失计数
0x212	DCache 请求计数
0x213	DCache 缺失计数
0x220	分支预测有效计数
0x221	分支预测错误计数
0x222	分支指令计数
0x223	条件分支指令计数
0x224	条件分支预测错误计数
0x225	FTB 污染计数
0x226	间接跳转指令计数
0x227	间接跳转预测错误计数
0x230	TLB 请求计数
0x231	TLB 缺失计数
0x232	TLB 重填状态周期计数

15 AXI 总线设计

内核采用 AXI128 与 SOC 上面的外设连接。AXI 总线使用 AXI4 的标准，将读写通道相互分离，支持同时读写主存。读数据和读指令分别采用两个结构相似的读状态机实现，这样能够保证 AXI 总线能够同时读数据和读指令，大大提高数据访问的速度，当 CPU 同时发出取值和取数的请求时，AXI 总线的 outstanding 值会变为 2。

15.1 读状态机



15.2 写状态机



15.3 仲裁器

现代 CPU 的设计宏观是冯诺依曼架构，微观上由于 dcache 和 icache 的存在是一个哈佛架构。CPU 的取指模块和访存模块都需要通过同一条 AXI 总线与主存进行数据交换，这样就出现了多个主设备同时访问同一个从设备的现象。因为总线一次只能处理一对主从设备的交互，所以当 CPU 同时发出取指和取数的操作时，需要进行仲裁，防止总线数据传送出现冲突。

15.3.1 写请求仲裁

由于向主存中写数据和读数据的速度不匹配，写需要花费较长的时钟周期，当写请求还没结束时，如果向主存发出读请求，很有可能会把写请求要写的地址处的旧的数值取出，导致出错。考虑到设计的简单性，AXI 总线在发出写请求后，必须要等到写请求结束后才能发出读请求。这样能够避免读到旧的数据。

15.3.2 读请求仲裁

1. 同时读指令和读数据

AXI 的读指令状态机和读数据相互分离。当同时发出读指令和读数据时，优先处理读数据请求，之后再处理读指令请求。由于读指令和读数据在两个状态机实现，此时的读数据和读指令可以看作是双通道并行处理。无随机延迟的情况下，读指令比读数据整体慢一个状态。

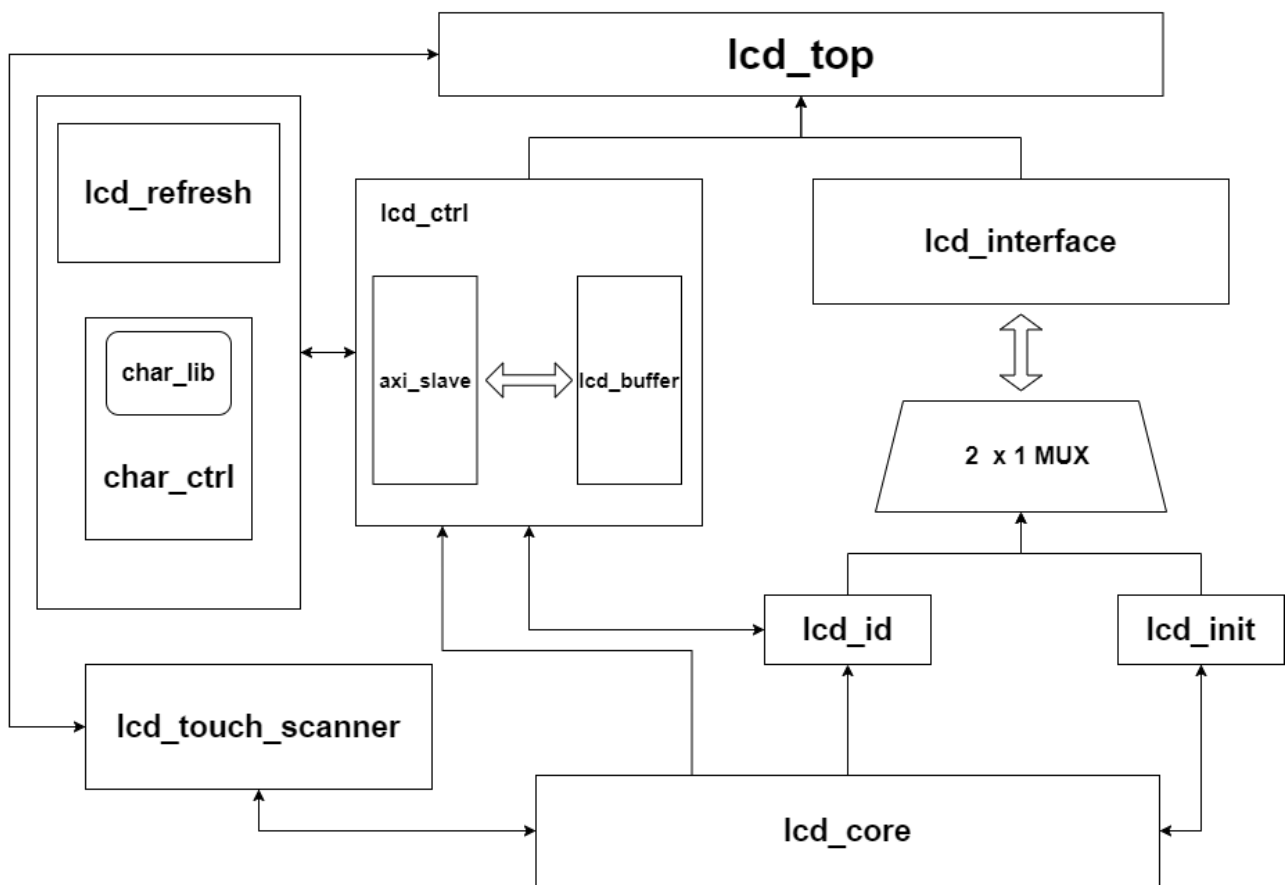
2. 先读指令后读数据

由于仲裁器是固定优先级的，当 AXI 中同时存在读数据请求和读指令请求（同时存在，不代表同时发出）时，都是默认读数据请求优先。考虑到逻辑上的合理性，当先发出读指令时，在处理完读指令请求前都不能处理读数据请求。这样能够最大限度的利用仲裁器，尽可能避免随机延迟时，读指令和读数据出现在 AXI 的同一个读通道上。

3. 先读数据后读指令

由于仲裁器固定优先级。在这种情况下，AXI 的仲裁策略和同时读指令和读数据是一样的。

16 系统展示



系统中实现了 LCD 控制器，用户可以通过 CPU 向 LCD 发送绘画的请求。同时 LCD 支持触碰功能，用户能够用手指或是触控笔触碰 LCD 屏幕，从而实现人机交互。此外 LCD 控制器还内置了字符库，用户可以绘制 ASCII 码中常用的字符。

16.1 LCD 控制器

SOC 集成了 LCD 控制器用于专门操作开发板上面的 LCD 显示屏。LCD 控制器实现了触摸和显示功能，用户能够通过 LCD 显示屏与系统实现实时的交互。LCD 控制器主要由初始化模块 **lcd_init**，cpu 交互模块 **lcd_ctrl**，译码模块 **lcd_id**，接口模块 **lcd_interface**，触摸模块 **lcd_touch_scanner** 和字符控制器 **char_ctrl** 等部件组成。LCD 控制器使用硬件的方式承担了大部分的重复的绘图任务，CPU 只需要传入少量的控制信号即可实现绘图。并且由于专门实现了用于和 CPU 交互的 **lcd_ctrl** 和对 CPU 信号进行译码的 **lcd_id** 模块，用户可以完全通过软件的方式实现 LCD 显示器的绘图。将 LCD 控制器的各项功能细分为不同模块，不同模块间的耦合度低，有利于拓展和移植。

16.1.1 LCD 显示模块

考虑操作 LCD 屏幕显示的基本原理就是不断的向 LCD 屏幕写入相应的指令和参数，为每一个像素点传入相应的颜色，从而实现绘图。如果每一条指令和参数都要由 CPU 单独通过 store 传入会严重影响系统的整体性能。在 LCD 屏幕上绘图的实际是朝一个矩形里面的像素点不断的传入颜色，对于 480*800 的屏幕而言如果通过 CPU 监控每一个像素点的绘制显然是不现实的。为了彻底解放 CPU，降低 LCD 显示对于 CPU 性能的影响，**lcd_ctrl** 模块中集成了硬件加速模块，CPU 绘制一张图像只需要传入 7 条 store 指令，用于设置坐标，颜色和图片大小，**lcd** 控制器即可自动的对这些 store 指令进行译码并写入 LCD 显示屏。在 **lcd** 控制器工作期间 CPU 无需等待 **lcd** 控制器工作完毕，可以去执行其他的任务。对于一些诸如开机 logo 显示，游戏页面初始化等重复性的固定操作，将完全通过硬件实现，无需 CPU 传入任何的绘图信息，极大的提高了 CPU 的工作效率。

16.1.2 LCD 触摸模块

触控部分移植了 20 年参赛队伍 UltraMips 的触摸模块代码。LCD 触摸屏幕初始化结束后，LCD 控制器将不断的监听屏幕上的触碰情况，将触摸点的信息放到专门的触碰信息寄存器 touch_reg 中，当 CPU 需要获取屏幕的触碰情况时，无需向 LCD 屏幕发出读写请求，只需要直接读取 LCD 控制器中内置的触控信息寄存器 touch_reg 即可，大大的提高了触碰信息的获取速度。同时，由于实现了触控功能，用户可以通过触碰 LCD 屏幕上不同选项，从而进行不同的游戏，使得 LCD 的显示更加的灵活。

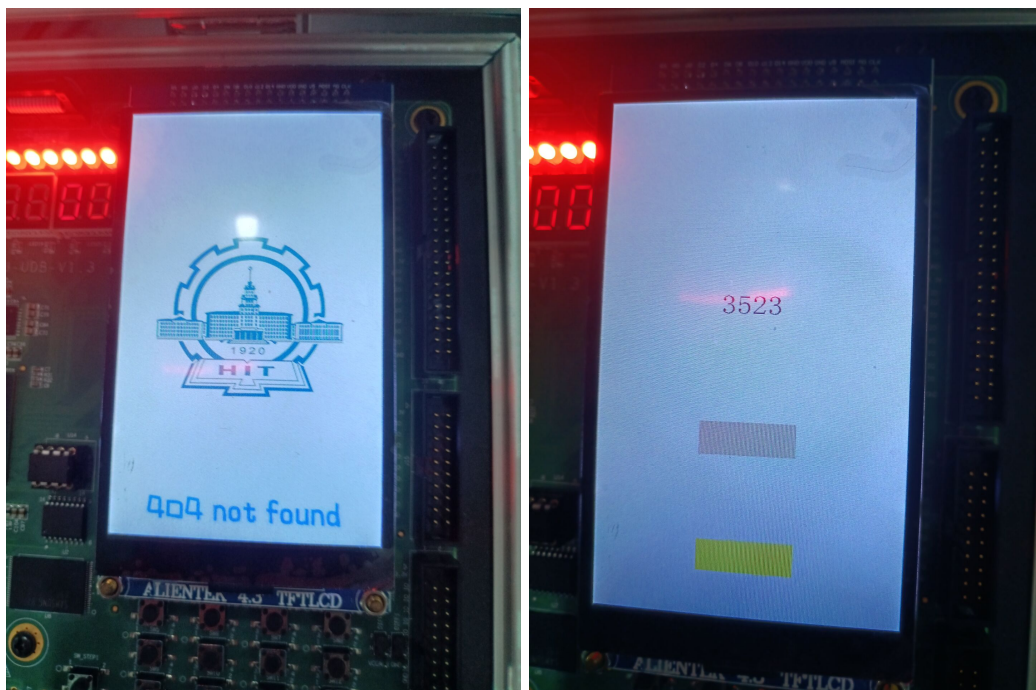
16.1.3 字符绘制模块

为了能够更加灵活的显示文字，使得系统展示的小游戏更加丰富，LCD 控制器中同时还集成了专门的字符控制器和字库。字库中存放着常用的数字和英文字符，以及队员的姓名。考虑到 FPGA 上资源有限，在保证美观的情况下，字符的大小统一为 24*40。CPU 只需要向 LCD 控制器中传送字符的大小位置以及字符选择信号，硬件即可自动从字库中取出相应的字符点阵，自动监控每个像素点的绘制，在 LCD 控制器绘制字符的过程中，CPU 可以执行其他操作，实现了外设和内核的并行工作。

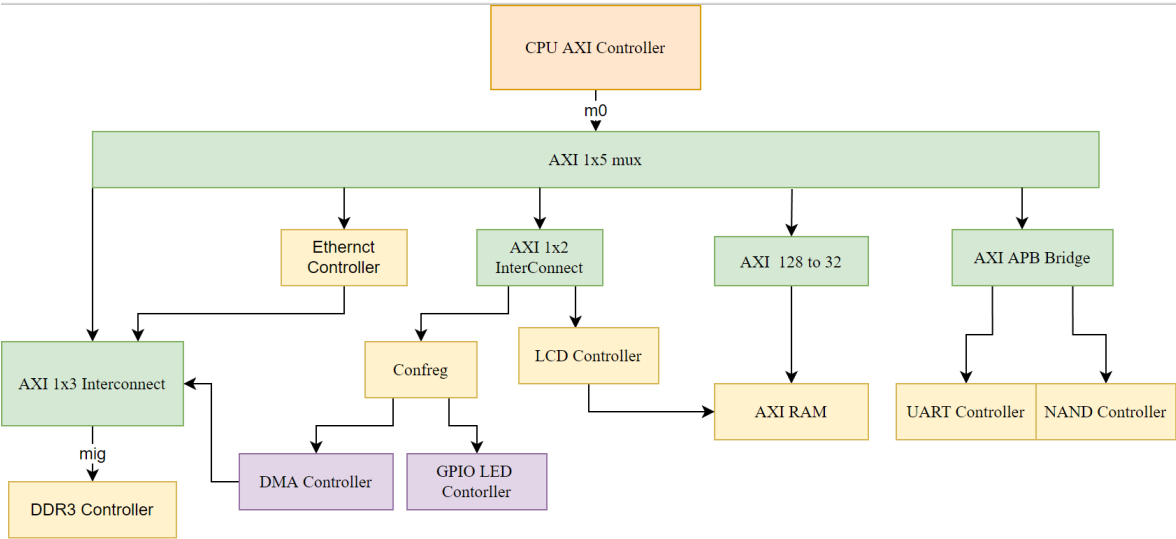
16.2 软件展示

CPU 和外设之间可以实现实时交互，用户能够通过运用 C 语言操作 LCD 的显示和绘制。LCD 目前集成了矩形动画，页面刷新，方块变色，随机数生成等几个小游戏。游戏的运行由软件监控，体现了 LCD 良好的拓展性，同时说明 CPU 具有拓展各类外设的能力。由于时间的关系，无法制作更多有意思的小游戏，但通过在 CPU 上面运行 C 语言从而操作外设的基本原理都一样。上面几个小游戏虽然简单，但是已经能够说明本系统具有充足的潜力去实现复杂多样的系统展示功能。

同时 LCD 自带一个简易的开机 LOGO，按下复位键后 LCD 会先显示本校校徽和队伍名字，之后进入主页面。在主页面下用户能够进行游戏选择。



16.3 系统展示 soc



17 性能记录

性能测试中选取的 baseline 成绩是由 chiplab 中的示例内核跑出的
示例内核的有关参数：33MHz, 8K L1i, 8K L1d, BTB

17.1 Coremark 成绩

baseline: 58.145169

时间	版本	分数
2022-07-19	33MHz, 64K L1i, 128K L1d, write-through, no BPU	24.960479
2022-08-01	66MHz, 32K L1i, 16K L1d write-through, 256K L2d, TAGE-predec	105.685902
2022-08-01	66MHz, 32K L1i, 16K L1d write-through, 256K L2d, TAGE-predec-ras	110.265740
2022-08-03	66MHz, 32K L1i, 16K L1d write-through, 256K L2d, TAGE-predec-ras-tuned	118.287632
2022-08-10	100MHz, 32K L1i, 8K L1d write-back, TAGE-predec-ras-tuned	198.775634
2022-08-14	70MHz, 32K L1i, 16K L1d write-through, 512K L2mixed, TAGE-predec-ras-tuned	128.324430

最终的提交版本因为一致性问题没有解决，使用了 write-through 的 L1d Cache。

17.2 Fireye 测试套件成绩

baseline: 1.64336

时间	版本	分数
2022-07-19	33MHz, 64K L1i, 128K L1d, write-through, no BPU	0.972231
2022-08-01	66MHz, 32K L1i, 16K L1d write-through, 256K L2d, TAGE-predec	4.30564
2022-08-01	66MHz, 32K L1i, 16K L1d write-through, 256K L2d, TAGE-predec-ras	4.43107
2022-08-03	66MHz, 32K L1i, 16K L1d write-through, 256K L2d, TAGE-predec-ras-tuned	4.79011
2022-08-11	70MHz, 32K L1i, 32K L1d write-through, 512K L2mixed, TAGE-predec-ras-tuned	5.36443
2022-08-14	70MHz, 32K L1i, 16K L1d write-through, 512K L2mixed, TAGE-predec-ras-tuned	4.94036

17.3 Fireye 测试套件分支预测正确率

Test Case	All Branches	Conditional	Unconditional
A0-1	99.811	99.943	73.328
A1-1	96.065	96.28	67.411
B0-10	86.71	83.376	98.717
B2-50	98.418	99.254	94.288
B3-50	96.267	97.265	93.981
B5-50	99.872	99.985	75.981
D0-100	87.35	83.648	97.879
D0-1000-10	99.405	99.908	98.181
D0-1000-06	98.753	99.144	94.384
F0-1	89.975	89.809	90.871
G0-10-03	95.711	98.379	88.448
G0-10-06	96.423	99.27	88.82
H0-40	91.092	93.354	74.482
I2-10	99.493	99.59	97.285
coremark-0	95.83	96.128	93.324
coremark-1	95.91	96.226	93.247
dhrystone	90.674	99.95	72.206

18 RoadMap

时间节点	目标
5 月下旬	实现 FTQ 和简易的 IFU，能够接入测试 (ICache 联合调试)
6 月上旬	接入 TLB
6 月中旬	完成其他部分的设计与测试
6 月下旬	加入 uBTB 的一级分支预测
7 月上旬	实现 TAGE 和 FTB
7 月	根据性能测试进一步优化 (指令预取、预译码、RAS 等)
8 月	修改访存微架构，解决 load to use 和主频问题

19 文档更新记录

时间	更新内容
2022-05-15	初始版本
2022-05-21	增加 ICache 设计
2022-05-24	完成基本的前端设计，增加设计图
2022-05-31	修改完善前端设计
2022-06-10	增加后端预测块提交的文档
2022-06-11	增加 TLB 相关文档
2022-06-25	修改了 ICache 设计
2022-07-05	重新设计了访存部分
2022-07-20	增加 PMU
2022-08-01	增加性能测试成绩
2022-08-03	增加分支预测准确率
2022-08-17	加入 RAS 和预译码，完善剩余流水级